Here is the **Final, Ultimate Software Requirements Specification (SRS)** for your **Smart Printing Module**.

This version (v6.0) is the "Gold Standard." It incorporates **Multiple Shop Support**, **Live Status Indicators (Open/Closed)**, **Hybrid Payments**, and **IoT Automation**, perfectly aligned with Microsoft Imagine Cup requirements.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Module Title: Smart Printing & Document Services ("InstaPrint") **Parent Project:** UniVerse: The Student Experience Super-App **Version:** 6.0 (Final Architecture) **Tech Stack:** React.js (PWA), Python (IoT Agent), Microsoft Azure **Primary Objective:** To create a decentralized "Uber-for-Print" marketplace connecting students with multiple campus print shops via automation.

---

1. Introduction

1.1 Purpose

The purpose of the **InstaPrint Module** is to eliminate physical queues and cash friction at university reprography centers. Unlike traditional models where students crowd one shop, this system routes print jobs to **multiple available shops** based on location, price, and real-time operational status (Open/Closed).

1.2 Scope

1. **Multi-Vendor Marketplace:** Students can view and select from various print shops (Library, Hostel, Main Gate).

2. **Live Status Tracking:** Real-time visibility of shop availability (Open/Closed/Busy) and resource status (Out of Paper).
3. **IoT Automation:** A desktop agent that auto-configures physical printers without human intervention.
4. **Sustainability:** AI-driven nudges to reduce paper waste.

---

2. User Classes and Characteristics

User Role	Description	Key Needs
Student	The end-user.	Wants to find the nearest *open* shop, upload files remotely, and pay via UPI.
Shop Owner	Independent vendor.	Wants to manage opening hours, automate print settings, and secure payments.
System Admin	You (The Startup).	Wants to onboard new shops and track global commission revenue.

---

3. Functional Requirements (Detailed)

3.1 Student Portal (Web/PWA)

FR-01: Shop Discovery & Live Status (Crucial Feature)

- **Map/List View:** System displays a list of all onboarded shops sorted by proximity or price.
- **Real-Time Status Indicators:**
  - 🟢 **OPEN:** Accepting orders immediately.
  - 🔴 **CLOSED:** Shop is offline; ordering disabled.

    - 🟡 **BUSY:** High queue load (>10 mins wait).
    - ⚪ **RESOURCE ALERT:** E.g., *“Library Shop: No Color Printing Available.”*

FR-02: Drag-and-Drop Cloud Upload

- **Description:** Supports PDF, DOCX, IMG uploads (Max 25MB).
- **Validation:** Generates a thumbnail preview to ensure the correct file is being printed.
- **Security:** Files are encrypted in **Azure Blob Storage** and auto-deleted after 24 hours.

FR-03: Smart Configuration & Eco-Nudge

- **Settings:** Toggle switches for **Color/BW**, **Single/Double Sided**, and **Copies**.
- **Sustainability Logic:** If “Single-Sided” is selected, a prompt appears: *“Switch to Double-Sided to save ₹X and trees.”*

FR-04: Hybrid Payment Gateway

- **Direct UPI Intent:** Integration with GPay/PhonePe for exact-amount payments (no wallet loading required).
- **Campus Wallet:** One-tap payment for power users.
- **Logic:** Print job is routed to the specific **Target Shop ID** only after payment success.

FR-05: Collection Security

- **OTP Generation:** A large 4-digit code (e.g., **#9090**) is generated for the student to claim the document.

3.2 Shop Owner Dashboard (Web + IoT Agent)

FR-06: Availability Management (The Toggle)

- **Master Switch:** A prominent **[GO ONLINE / GO OFFLINE]** toggle on the dashboard header.
- **Auto-Offline:** System automatically marks a shop as "Offline" if the Shop PC loses internet connection for >5 minutes.
- **Resource Toggles:** Owners can disable specific services (e.g., *"Toggle OFF A3 Paper"*), which instantly updates the Student App.

FR-07: Live Kanban Queue

- **Columns:** Incoming → Printing → Ready for Pickup.
- **Visual Priority:** Jobs are color-coded (Red Border = Color Print) for instant recognition.

FR-08: IoT Automation ("One-Click" Print)

- **Architecture:** The Web Dashboard sends a signal to the specific **Shop PC** via **Azure IoT Hub**.
- **Action:** The local **Python Script** downloads the file → Sets Driver (Color/Duplex) → Prints.
- **Benefit:** Eliminates human error (e.g., accidentally printing Color for a B/W order).

FR-09: Manual Fail-Safe

- **Fallback:** If the automation script fails, a "Manual Download" button allows the owner to print using standard OS dialogs.

---

4. Non-Functional Requirements (NFRs)

- **NFR-01: Latency:** Status updates (e.g., Shop going Offline) must reflect on the Student App within **5 seconds** (using WebSockets/SignalR).
- **NFR-02: Accessibility:** Interface must support Screen Readers and High Contrast modes for visually impaired students.
- **NFR-03: Scalability:** The backend must support routing jobs to 50+ distinct shop IDs simultaneously.
- **NFR-04: Reliability:** Payment reconciliation must be 100% accurate; no job is lost if payment is deducted.

---

5. Technical Architecture (Azure Stack)

The system follows a **Multi-Tenant Microservices Architecture**:

1. **Frontend:** React.js (PWA) with Location Services API.
2. **Backend:** Node.js (Azure Functions).
3. **Database: Azure Cosmos DB** (Stores `Shops Collection` with status flags and `Jobs Collection`).
4. **IoT Layer: Azure IoT Hub** (Manages "Device Twins" for each Shop PC).
5. **Local Agent:** Python Script using `win32print` library.

---

6. User Interaction Flow

1. **Student:** Opens App $\rightarrow$ Sees "Library Shop" is **OPEN** $\rightarrow$ Selects it.
2. **Student:** Uploads File $\rightarrow$ Selects Config $\rightarrow$ Pays via UPI.
3. **Cloud:** Routes job specifically to **Library Shop Device ID**.
4. **Library Shop PC:** Python script wakes up $\rightarrow$ Auto-prints document.

5. **Shopkeeper:** Staples document → Writes OTP on it → Places in rack.
6. **Student:** Arrives → Shows OTP → Collects.

---

7. Competitive Advantage (For Imagine Cup)

Feature	Competitor (Standard Print)	UniVerse InstaPrint
Shop Choice	Single location (Crowded)	**Multi-Shop Marketplace**
Availability	Walk there to check	**Live Status on App**
Setup	Manual (Error prone)	**IoT Auto-Configuration**
Payment	Cash/Change issues	**Exact UPI / Wallet**
Sustainability	High Waste	**AI-Driven Eco Nudges**

Final Note for Implementation:

- **Database Tip:** In your Cosmos DB `Shops` collection, ensure you have a field `isActive: boolean`. Your student app query should be `SELECT * FROM Shops WHERE isActive = true` to show open shops.
- **IoT Tip:** Use **Device IDs** (e.g., `shop_01_printer`) in Azure IoT Hub to ensure the print command goes to the right building.